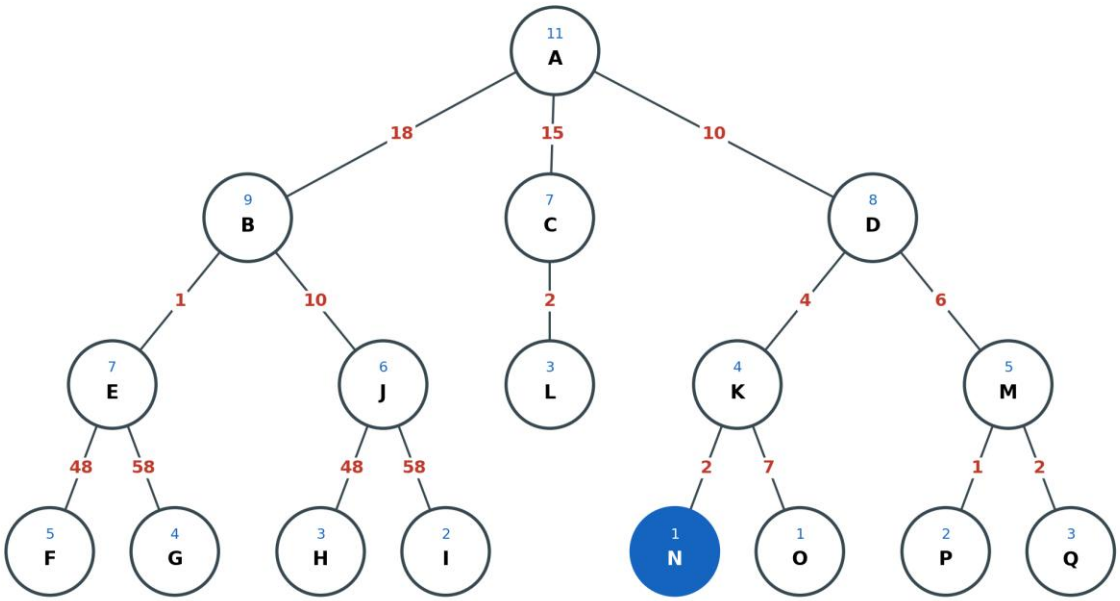
CAI104 Concepts in AI
Assessment 2: Workbook

Student: _____

Question 1: Search Methods on the State Space Tree (30%)

The state space tree used for this question is shown below. The number inside each circle is the actual distance to the target (used as the heuristic h); the number on each edge is the step cost g.



Goal node = N (blue). Inside circle = h (distance to target). Edge label = step cost g.

Goal node = N (blue). Children are expanded left-to-right and the goal test is applied when a node is expanded (removed from the frontier).

Tabulated values used by the informed searches (g = cost from root, h = number inside the circle, f = g + h):

Node	g	h	f = g+h
A	0	11	11
B	18	9	27
C	15	7	22
D	10	8	18
E	19	7	26

J	28	6	34
L	17	3	20
K	14	4	18
M	16	5	21
F	67	5	72
G	77	4	81
H	76	3	79
I	86	2	88
N	16	1	17
O	21	1	22
P	17	2	19
Q	18	3	21

1.1 Breadth-First Search (BFS)

BFS expands the shallowest unexpanded node, exploring the tree level by level (left to right) using a FIFO queue.

Order of traversal:

```
A -> B -> C -> D -> E -> J -> L -> K -> M -> F -> G -> H -> I -> N
```

Goal found: Yes. BFS reaches N on level 3 after expanding 14 nodes.

1.2 Depth-First Search (DFS)

DFS expands the deepest unexpanded node first, following the left-most branch to a leaf before backtracking (LIFO stack).

Order of traversal:

```
A -> B -> E -> F -> G -> J -> H -> I -> C -> L -> D -> K -> N
```

Goal found: Yes. DFS reaches N after expanding 13 nodes.

1.3 Iterative Deepening Depth-First Search (IDDFS)

IDDFS runs depth-limited DFS with the limit increasing from 0 until the goal is found, combining DFS's low memory with BFS's completeness.

Iteration by depth limit:

```
L = 0:  A                                              (goal not found)
L = 1:  A -> B -> C -> D                                (goal not found)
L = 2:  A -> B -> E -> J -> C -> L -> D -> K -> M       (goal not found)
L = 3:  A -> B -> E -> F -> G -> J -> H -> I ->
        C -> L -> D -> K -> N                           (GOAL FOUND)
```

Goal found: Yes, at depth limit 3 (the depth of N).

1.4 Greedy Best-First Search

Greedy best-first search always expands the frontier node with the smallest heuristic h(n), ignoring the cost already paid.

Step-by-step (frontier shown with h values):

```
Expand A   -> frontier: B(9), C(7), D(8)          pick C (min h=7)
Expand C   -> frontier: B(9), D(8), L(3)          pick L (min h=3)
Expand L   -> frontier: B(9), D(8)                pick D (min h=8)
Expand D   -> frontier: B(9), K(4), M(5)          pick K (min h=4)
Expand K   -> frontier: B(9), M(5), N(1), O(1)    pick N (min h=1)
Expand N   -> GOAL FOUND
```

Order of traversal:

```
A -> C -> L -> D -> K -> N
```

Goal found: Yes (6 nodes), but the A-C-L detour shows greedy is not guaranteed to be optimal.

1.5 A* Search

A* expands the frontier node with the smallest f(n) = g(n) + h(n), balancing cost paid (g) against estimated cost remaining (h).

Step-by-step (frontier shown with f values):

```
Expand A  -> frontier: B(27), C(22), D(18)              pick D (f=18)
Expand D  -> frontier: B(27), C(22), K(18), M(21)       pick K (f=18)
Expand K  -> frontier: B(27), C(22), M(21), N(17), O(22)  pick N (f=17)
Expand N  -> GOAL FOUND
```

Order of traversal:

```
A -> D -> K -> N
```

Goal found: Yes (only 4 nodes). Path A-D-K-N has cost g = 10 + 4 + 2 = 16, the cheapest route, so A* is optimal.

Discussion: Efficiency and Applications

Comparison on this problem:

Method	Nodes expanded	Finds N?	Optimal path?
BFS	14	Yes	Yes (fewest edges)
DFS	13	Yes	Not guaranteed
IDDFS	1+4+9+13	Yes	Yes (unit depth)
Greedy	6	Yes	No
A*	4	Yes	Yes (cost = 16)

For this problem A* is the most efficient: it expands the fewest nodes (only 4) and still returns the optimal-cost path because the heuristic (the true distance to the target) is admissible and consistent. Greedy expands few nodes too, but it is lured down the C-L branch and returns a sub-optimal route, trading correctness for speed. The uninformed methods (BFS, DFS, IDDFS) ignore the heuristic and therefore expand many more nodes.

Applications. BFS suits unweighted shortest-path problems such as finding the fewest hops in a social network or the minimum moves in a puzzle. DFS is memory-cheap and is used for

topological sorting, cycle detection, maze generation and constraint-satisfaction backtracking. IDDFS is the practical choice when the search depth is unknown and memory is tight, as in game-tree search. Greedy best-first powers fast but approximate routing where speed matters more than optimality. A* is the workhorse of optimal pathfinding in GPS navigation, robotics and video-game movement, because a good admissible heuristic lets it reach the goal while exploring only a small, well-directed part of the space.

Question 2: Propositional Logic and Truth Tables (30%)

Reference truth table for the basic connectives:

P	Q	¬P	P∧Q	P∨Q	P⇒Q	P⇔Q
T	T	F	T	T	T	T
T	F	F	F	T	F	F
F	T	T	F	T	T	F
F	F	T	F	F	T	T

2.1 $(p \Rightarrow q) \wedge (p \Rightarrow r) \Leftrightarrow p \Rightarrow (q \wedge r)$

p	q	r	p⇒q	p⇒r	(p⇒q)∧(p⇒r)	q∧r	p⇒(q∧r)	⇔
T	T	T	T	T	T	T	T	T
T	T	F	T	F	F	F	F	T
T	F	T	F	T	F	F	F	T
T	F	F	F	F	F	F	F	T
F	T	T	T	T	T	T	T	T
F	T	F	T	T	T	F	T	T
F	F	T	T	T	T	F	T	T
F	F	F	T	T	T	F	T	T

The final column (⇔) is true in every row, so the two sides are logically equivalent — the statement is a tautology (valid).

2.2 $(p \vee q) \wedge (\neg p \vee q) \Rightarrow q$

p	q	¬p	p∨q	¬p∨q	(p∨q)∧(¬p∨q)	⇒q
T	T	F	T	T	T	T
T	F	F	T	F	F	T
F	T	T	T	T	T	T
F	F	T	F	T	F	T

The final column is true in every row — the implication is a tautology (valid). This is the resolution rule.

2.3 Logical consequence: $A = \{ p \Rightarrow q,\ q \Rightarrow p,\ p \vee q \},\ C = p \wedge q$

p	q	p⇒q	q⇒p	p∨q	All premises	C = p∧q
T	T	T	T	T	TRUE	T
T	F	F	T	T	false	F
F	T	T	F	T	false	F
F	F	T	T	F	false	F

Only the row p=T, q=T satisfies all three premises, and there C is also true. Therefore $A \models C$: C is a logical consequence of A.

2.4 Logical consequence: $A = \{ q \vee r,\ q \Rightarrow p,\ \neg(r \wedge p) \},\ C = \neg p$

p	q	r	q∨r	q⇒p	r∧p	¬(r∧p)	All prem.	C = ¬p
T	T	T	T	F	T	F	false	F
T	T	F	T	F	F	T	false	F
T	F	T	T	T	T	F	false	F
T	F	F	F	T	F	T	false	F

F	T	T	T	T	F	T	TRUE	T
F	T	F	T	T	F	T	TRUE	T
F	F	T	T	T	F	T	TRUE	T
F	F	F	F	T	F	T	false	T

The premises are all true in rows 5, 6 and 7, and in each of those rows C = ¬p is true. Therefore A ⊨C: ¬p is a logical consequence of A.

Discussion: Applications of Logic

Propositional logic lets us encode knowledge as statements that are definitely true or false and then reason about them mechanically, which is exactly what digital hardware, rule engines and verification tools do. A concrete example mapped onto the variables above: let p = “the motion sensor is triggered”, q = “the alarm sounds” and r = “the security company is notified”. Then a premise set like A in 2.4 expresses house rules such as “the alarm only sounds when the owner is away”, and a conclusion C = ¬p (“no intrusion”) can be derived automatically. The same machinery validates digital circuits, checks software with SAT solvers, drives expert-system rule bases in medical diagnosis, and underpins database query optimisation — anywhere we need guaranteed, repeatable conclusions from a fixed set of facts and rules.

Question 3: Translation into First-Order Logic (20%)

Predicates: Scientist(x), Smart(x), Loves(x, y), Publishes(x, y); with Politics, Articles, Books as constants.

1. All scientists are smart.

$\forall x \; (\; Scientist(x) \Rightarrow Smart(x) \;)$

2. There exists a smart scientist.

$\exists x \; (\; Scientist(x) \wedge Smart(x) \;)$

3. No scientist loves politics.

$\forall x \; (\; Scientist(x) \Rightarrow \neg \; Loves(x, Politics) \;)$
$\quad$ equivalently: $\neg \; \exists x \; (\; Scientist(x) \wedge Loves(x, Politics) \;)$

4. Every scientist who publishes articles also publishes books.

$\forall x \; (\; (\; Scientist(x) \wedge Publishes(x, Articles) \;)$
$\qquad \Rightarrow Publishes(x, Books) \;)$

Question 4: Translation and Validity of Reasoning (20%)

4.1

"If I'll go to the university then I'll take a taxi, or if I'll take a taxi then I'll go to the university."

Let u = "I go to the university" and t = "I take a taxi". The sentence is the disjunction of two implications:

$(u \Rightarrow t) \vee (t \Rightarrow u)$

u	t	u⇒t	t⇒u	(u⇒t)∨(t⇒u)
T	T	T	T	T
T	F	F	T	T
F	T	T	F	T
F	F	T	T	T

The final column is true in every row, so the sentence is a tautology — always valid regardless of the facts.

4.2

"It is not true that: Alex bought a car, or if he bought a car then he had money. Therefore Alex didn't have money."

Let c = "Alex bought a car" and m = "Alex had money".

- **Premise:** $\neg (c \vee (c \Rightarrow m))$
- **Conclusion:** $\neg m$

c	m	c⇒m	c∨(c⇒m)	Premise ¬(…)	Conclusion ¬m
T	T	T	T	F	F
T	F	F	T	F	T
F	T	T	T	F	F
F	F	T	T	F	T

The premise $\neg(c \vee (c \Rightarrow m))$ is FALSE in every row, because $c \vee (c \Rightarrow m)$ is itself a tautology. A premise that can never be true is a contradiction.

Validity verdict:

The argument is technically valid in the vacuous sense — there is no row where the premise is true and the conclusion false, simply because the premise is never true. However it is NOT sound: the premise is a contradiction, so the conclusion ¬m does not genuinely follow from any real situation, and the reasoning is unusable in practice.

Discussion: Applications of Reasoning

Reasoning is the process of deriving new, justified conclusions from what is already known, and it is the engine behind every knowledge-based AI system. Deductive reasoning of the kind shown above guarantees that true premises yield true conclusions, which is essential where errors are costly: automated theorem provers and formal verification prove that aircraft and chip designs cannot fail; expert systems in medicine and law chain rules to reach diagnoses or

judgements; and semantic-web reasoners infer implicit facts from ontologies. Question 4.2 also shows why systems must check soundness, not just validity: a knowledge base containing a contradiction can “prove” anything, so consistency checking matters. Beyond strict deduction, AI also uses inductive reasoning (generalising from data, as in machine learning) and abductive reasoning (inferring the most likely explanation, as in fault diagnosis), giving agents a full toolkit for acting sensibly under both certainty and uncertainty.

— End of Workbook —